SMARTSME: A CONFIRMATION-FIRST, EVENT-DRIVEN PLATFORM FOR INFORMAL ORDER CAPTURE AND BOOKKEEPING IN SMALL AND MEDIUM ENTERPRISES

[1][AUTHOR NAME], [2][AUTHOR NAME], [3][AUTHOR NAME], [4][AUTHOR NAME]

[Department], [Institution], [City], India
E-mail: [author email addresses]

Abstract - Micro, small and medium enterprises (MSMEs) contribute about 30% of India's gross domestic product, yet many of them still receive orders through chat messages, telephone calls and handwritten slips that are never entered into a ledger capable of being reconciled with stock or credit. This paper presents SmartSME, a web-based business management platform designed to capture such orders without requiring customers to change how they place them. A Smart Input engine, which combines a hosted language model with an offline rule-based parser, converts typed notes and photographed order slips into draft sales, purchases and expenses, and no draft is recorded until a user has confirmed it. Each confirmed write is committed atomically with its event through a transactional outbox in PostgreSQL, from which a configurable rule engine derives the corresponding stock, balance and alert effects. On 70 synthetic typed notes, the offline parser achieved an exact-match accuracy of 87.1% and the language model 97.1%. On 12 real handwritten slips, a vision model achieved 100%, compared with 75.0% for character-level OCR, whose errors were confined to slips containing struck-out items. On a separate set of 24 notes written outside the system's vocabulary, the offline parser produced fully correct drafts for 29.2%; in an earlier run, the language model handled unfamiliar Hindi and Kannada grammar far better, and most of the remaining errors arose in matching product names to the catalogue. Under a load of 200 simulated users, throughput increased from 3.5 to 106.3 requests per second once three bottlenecks had been removed. The limitations of the evaluation data are examined, and field validation is recommended before wider deployment.

Keywords - Small and medium enterprises, Event-driven architecture, Transactional outbox, Information extraction, Code-mixed text, Optical character recognition.

I. INTRODUCTION

Micro, small and medium enterprises (MSMEs) account for a large share of Indian economic activity. According to official estimates, the sector's gross value added amounted to 30.1% of all-India GDP in 2022-23, and MSME-related products made up 45.73% of the country's exports in 2023-24 [1]. Small unincorporated units are especially numerous. The Annual Survey of Unincorporated Sector Enterprises counted 7.34 crore unincorporated non-agricultural establishments in 2023-24, which together employed more than 12 crore workers [2]. The Micro, Small and Medium Enterprises Development Act of 2006 gives the sector a distinct legal identity and protects its suppliers against delayed payment by buyers [3].

Digitisation inside these firms has been uneven. In a survey of 2,882 MSMEs published in 2025, 87% made and received payments digitally, yet only 12% used enterprise resource planning (ERP) software [4]. Payments, in other words, have gone digital far faster than the software that runs the rest of the business. Orders still come in as a chat message asking for ten bags of rice, or as a handwritten slip passed across the counter, and none of them reaches an accounting system.

Packaged ERP and accounting products expect data to be entered through forms, and studies of their uptake among small firms repeatedly cite cost, implementation effort and staff training as barriers [5, 6]. When the owner must key in every transaction at the end of the day, entries are postponed or forgotten, and the recorded stock, receivables and payables slowly diverge from reality. The shortfall lies in the mismatch between the way orders arrive and the way such software expects to receive them.

Two technical problems follow. Informal input has to be converted into structured records reliably, even when a note mixes Hindi or Kannada with English, uses non-standard spellings or arrives as a photograph of handwriting. Recorded transactions also have to stay consistent with their effects on inventory, party balances and alerts, and a small business should be able to achieve this without operating a separate message broker.

This paper makes four contributions:

1. A Smart Input engine in which one parser handles the in-app text window, photographed slips, email and Telegram. Each extraction is matched against the shop's own catalogue in Latin, Devanagari or Kannada script, and nothing is written until a person confirms the draft.
2. An event layer that applies the transactional outbox pattern in PostgreSQL, with an optional Redis and Celery dispatch mode under which delivery remains at-least-once and application exactly-once.
3. A WHEN/THEN rule engine that users can configure, together with a business-health score computed from a

dated payment ledger, whose components follow stated formulas.

4. A measured evaluation of extraction accuracy, including a test on vocabulary the system was not built for, together with run-to-run consistency, latency and throughput, reported alongside the cases in which the system fails.

Section II describes the operational context of a small trading business, and Section III sets out the challenges of digitising its records. Existing approaches are reviewed in Section IV. Section V then describes the design of SmartSME, which Section VI evaluates. Limitations are discussed in Section VII, and Section VIII concludes the paper.

II. OPERATIONAL CONTEXT OF A SMALL TRADING BUSINESS

Four functional areas are enough to describe a typical small trading business, such as a neighbourhood grocery (kirana) store or a local wholesaler. Because each area feeds the others, a mistake in one spreads to the rest.

1. *Order capture:* Orders come from walk-in customers, phone calls, chat applications, email and paper slips. One order can contain several items, refer to the customer by an informal name ("Latha" for "Lathamma Provisions"), and give a date relative to when it was written, as with the Hindi "kal", which means either yesterday or tomorrow.
2. *Inventory:* Purchases from suppliers add to stock and sales to customers reduce it. An item that runs out costs sales at once, and one that sells slowly ties up cash the shop could use elsewhere. Stock figures are only as accurate as the record of sales and purchases behind them.
3. *Credit and payments:* Credit runs both ways. Customers pay some time after buying, and suppliers are paid against bills on agreed terms. Late payment troubles small Indian suppliers enough that the MSMED Act caps the payment period a buyer can agree with a micro or small supplier at 45 days [3]. Tracking who owes how much, and for how long, is possible only if every payment is recorded with its date.
4. *Expenses and reporting:* Overheads consist of rent, salaries, utilities and transport. Combined with sales and the cost of goods sold, they decide whether the business makes a profit and whether its cash position is getting better.

The links between these areas, and the points at which informal channels feed into them, are shown in Figure 1.

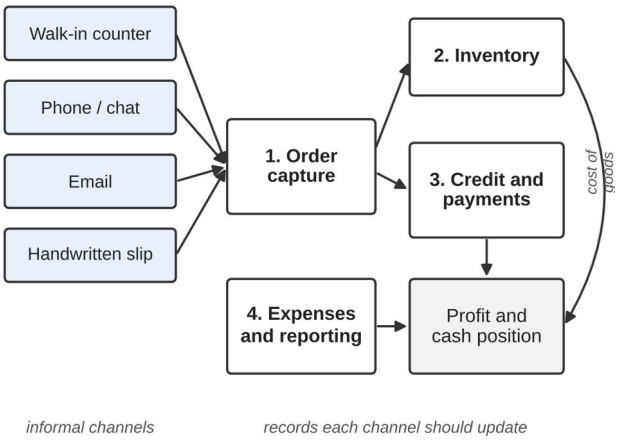


Figure 1: The four functional areas of a small trading business and the informal order channels that feed them

III. CHALLENGES IN DIGITISING INFORMAL SME RECORDS

Turning informal input into a ledger that can be trusted involves problems of language as well as of system design.

A. Language, script and spelling

In Karnataka, a single sentence of an order may draw on Kannada, Hindi and English, written in Kannada script, Devanagari or Latin letters. Such code-mixing is widespread in Indian online communication [7], and code-switching benchmarks report weaker model performance on it than on monolingual text [8]. Romanised Indian words also lack a fixed spelling, so "chawal", "chaval" and "chaawal" all refer to rice, and a customer may be written as "Anita" or "Anitha". Grammar adds a further complication. Hindi and Kannada place the marker for a recipient after the name ("Anita ko", "Anitage"), whereas English places a preposition before it.

B. Handwriting and in-place corrections

People correct paper slips on the spot, striking out an item and writing another beside it. Character-level OCR engines such as Tesseract recognise characters and lines [9], and a strikethrough has no character to recognise. If a struck-out item is read as part of the order, the customer is billed for it.

C. Reliability of model output

Large language models are able to pull structured fields out of free text given few or no examples [10], but they also generate fluent output that the input does not support [11]. Their output is sampled, so repeated runs on one input may disagree. In a ledger, a blank customer field is easy to spot and fill in manually. A wrong name does more harm, since it bills someone who never placed the order.

D. Consistency of records and their effects

Every sale has to lower the stock count and change the customer's balance. When the sale is stored in a database and the notification of it is sent separately through a message broker, either step can fail without the other,

which is the well-known dual-write problem. Distributed transactions that span both systems are expensive and fragile [12], and a small business has no operations staff to repair inconsistencies after they appear.

E. Operating constraints

Small businesses have little money for software and rarely any technical staff. A hosted AI service therefore has to be optional, and the free tier of the model used here allows roughly fifteen requests per minute. The system must keep working when the service is slow, down or simply not configured.

IV. EXISTING APPROACHES

This section reviews the techniques that SmartSME builds on or avoids, with the main advantages and drawbacks of each. The design in Section V follows from these assessments.

A. Packaged ERP and Accounting Software

ERP systems integrate sales, purchasing, inventory and accounting over a shared database, and their benefits for coordination are well documented [6]. Literature reviews of ERP in small and medium enterprises, however, report recurring barriers of cost, customisation effort and dependence on trained users [5]. These systems work well once the data has been entered, but entry still depends on typing each transaction into a form, which is the step informal trade leaves out.

B. Message-Broker Event-Driven Architectures

Components in an event-driven architecture interact by publishing and consuming events instead of calling each other directly [13, 14]. With a broker such as RabbitMQ between them, producers and consumers are decoupled and work can proceed asynchronously. The original SmartSME specification proposed this arrangement. However, the broker adds a further service to deploy and monitor, and by itself it does not solve the dual-write problem described in Section III-D.

C. Event Sourcing and the Transactional Outbox

In event sourcing, each change to application state is stored as an event in an append-only log [15]. The transactional outbox pattern is narrower in scope. It writes the business row and a record of its event to the same database in a single transaction and leaves delivery of the event to a separate relay [16, 17]. Neither can be committed alone, which removes the dual write. The price is that the database must also act as a queue, so polling, claiming and retrying require care.

D. Rule-Based Automation

An event-condition-action (ECA) rule, as studied in active database systems such as HiPAC [18], performs an action when a given event occurs and a condition holds. For large rule sets, production-rule systems based on the Rete algorithm provide efficient matching [19]. A shopkeeper can read such a rule and change it without touching code, which matters because policies such as the threshold for flagging an expense differ from shop to shop. The limitation is that a rule can only test fields the event actually carries.

E. Character-Level OCR

Mature OCR engines are inexpensive and accurate on clean printed text [9], and hosted services such as OCR.space offer free monthly quotas. What they return is plain text. Item rows have to be rebuilt from it afterwards, and marks such as strikethroughs are gone before any later stage could act on them.

F. Document-Understanding Models

Models that combine text with layout, such as LayoutLM [20], and OCR-free transformers, such as Donut [21], read forms and receipts with a structural awareness that character OCR lacks. Both usually require training data for the specific task and hardware to run on, and a single shop has neither.

G. Large Multimodal Models

General-purpose multimodal models such as Gemini accept both text and images and can follow instructions to return structured JSON without task-specific training [22]. This makes them a good fit for untidy notes and handwritten slips. Their drawbacks are the reliability issues discussed in Section III-C, dependence on a network connection and on the provider's terms of use, and the rate limits of free tiers.

H. Code-Mixed Processing and Transliteration

Studies of code-mixed Hindi-English text have described how speakers move between the two languages [7] and have supplied benchmarks for evaluation [8]. Deterministic transliteration libraries map Devanagari and Kannada script to Latin letters using lookup tables [23]. Transliteration keeps the sound of a word and loses its meaning. It can therefore establish that "अनीता" and "Anita" are one name, but not that "चावल" means rice.

I. Financial Health Scoring

Classical models of business distress, such as Altman's Z-score, combine financial ratios drawn from audited statements [24]. Few small shops maintain balance sheets, so any measure of their health must rely on the records they do keep, namely sales, purchases, expenses, stock and dated payments.

V. PROPOSED SYSTEM: SMARTSME

Four design criteria were drawn from this review. (1) Customers continue to order as they already do, and the system accepts plain text, photographs, email and chat accordingly. (2) Nothing from an external source is written to the books until a person confirms it. (3) No paid service is required, and the system keeps working, with reduced capability, when no AI provider is available. (4) One database is both the system of record and the event queue, which eliminates the dual write and the need for a broker.

A. Architectural Overview

SmartSME is structured as a modular monolith. Its backend is a FastAPI application that uses SQLAlchemy on top of PostgreSQL 16, with schema changes managed through Alembic migrations. The frontend, a React single-page application built with Vite and Tailwind, talks to it through a JSON API authenticated by cookie. Routers deal with HTTP parsing, permissions and serialisation and then pass each request to a domain layer, which contains the business rules and is unaware of HTTP. Tenants share one database and are separated by a business identifier on every row, which each route checks. Figure 2 shows the overall structure.

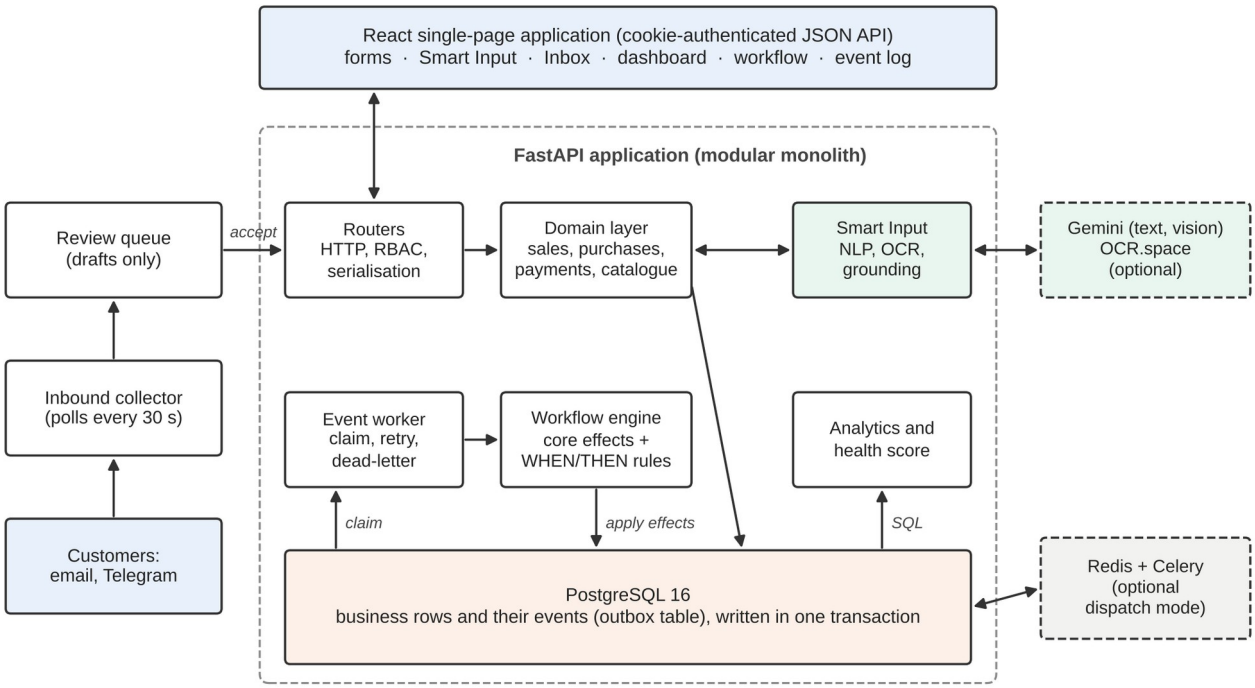


Figure 2: Overall architecture of SmartSME

Although the original specification called for a Node.js backend and a RabbitMQ broker, the system was built in Python with FastAPI. Following criteria (3) and (4), a transactional outbox took the place of the broker, and Redis with Celery was retained only as an optional dispatch mode.

B. Smart Input Engine

The Smart Input engine turns a free-text note or a photograph into a draft business event. It is shared by four entry points (the in-app Smart Input window, uploaded images, inbound email and Telegram messages), each of which yields the same draft object and goes through the same confirmation step. Figure 3 outlines the pipeline.

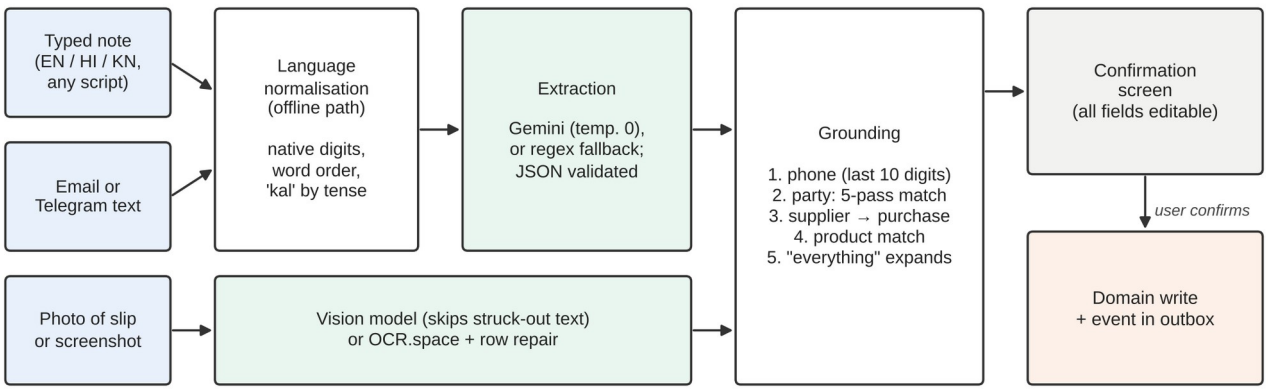


Figure 3: Smart Input pipeline from raw note or image to a confirmed business event (normalisation runs on the offline path only)

1. *Language normalisation:* The offline path, taken when no AI provider is configured or a call to it fails, begins with a module that rearranges Hindi and Kannada notes into the English word order expected by the built-in parser. Native digits (೫, ५) are converted to numbers, and postpositions are moved in front of the names they mark. The vocabulary is applied before suffixes are split, because some ordinary words happen to end in a

case marker. The Kannada word for rent (ಬಾಡಿಗೆ), for instance, ends in the same letters as the marker for "to", and splitting it would invent a customer called ಬಾಡಿ. The Hindi word "kal" is resolved by tense: a note about something already done refers to yesterday, and a request refers to tomorrow. Names and products stay in their original script; a later stage connects them to the catalogue.

2. *Extraction:* When a Google Gemini key is configured, a single prompt asks the model for strict JSON containing the event type, party, line items, amount, expense category, discount and date. Each line item carries the product as written in the note and, when that is not already English, the plain English name a shop's product list would use for it. The first JSON object in the reply is extracted and every field is validated. If no key is set, if the call fails, or if the reply cannot be parsed, a dependency-free regular-expression parser that covers the same fields is used instead. A provider that fails three times in a row is skipped for two minutes, so that a queue of mailed orders does not wait for a sixty-second timeout on each one. Model temperature is fixed at zero, for reasons given in Section VI-E.
3. *Multiple items:* A note frequently lists several goods, so line items are stored as a list. The regex parser splits on quantities rather than on the word "and". As a result, "2 kg salt and pepper" stays one item and "5 rice 2 sugar" becomes two, whereas splitting on conjunctions would get both cases wrong.
4. *Grounding:* The extracted names are then matched against the business's own records. For photographs, the phone number on the slip is compared first using its last ten digits, because it is the only exact key a slip carries. A slip signed "Latha" can belong to a shop filed as "Lathamma Provisions", a link that name matching would miss. When a matched party is a supplier, the draft becomes a purchase even if the model guessed a sale. Products are matched against the catalogue, and a phrase such as "everything" or "entire inventory" expands to one line per product in stock.
5. *Confirmation:* The draft, with its party, items, discount and date all editable, is presented to the user. Only an explicit confirmation passes it to the same domain functions that the ordinary forms use.

C. Cross-Script Matching

Matching "चावल" in a note to "Rice Bag 25kg" in a catalogue involves two different problems. Goods are described by different words across languages, which calls for a vocabulary. People's names are the same word in different scripts, which calls for transliteration. SmartSME treats the two separately. A retail lexicon of 94 concepts maps variants in every script and spelling to an English concept, and a deterministic transliteration library [23] converts names to Latin letters.

Both are then compared on a folded key that discards the features on which writers disagree, namely diacritics, doubled letters, aspirates ("th" and "t"), long vowels ("ee" and "i"), "w" and "v", and the inherent trailing vowel of Devanagari. For example, "chawal", "chaval", "chaawal" and "चावल" all fold to the same key. A best-match routine then proceeds through five passes, from certain to merely likely:

1. An exact match on the name.
2. The same words after folding removes differences of script and spelling.
3. The same concept by meaning, so that चावल, chawal and ಅಕ್ಕಿ all reach "rice". A word placed before the known one, such as "ragi" in "ragi atta", must also appear in the product's name, so that one kind of flour is not matched to another.
4. The same words, reordered.
5. The same consonants, accepted only when exactly one candidate shares them.

The last pass is needed because transliteration cannot restore a vowel the original script never wrote, so "स्टोर्स" comes back as "storsa". Being lossy, it answers only when a single candidate matches and otherwise returns nothing, leaving the field for the user to fill.

Machine translation was considered for product words and rejected. A dictionary runs offline with no delay and can be audited line by line, and it handles brand names, local produce names and trade terms such as "bori", which translation tends to get wrong. A translated word would also still need matching against the shop's catalogue, and that matching step is where the useful work happens.

Language support therefore differs between the two extraction paths. With a model configured, any Hindi or Kannada phrasing the model understands can be extracted, and when a product word finds nothing in the catalogue the matcher tries the model's English name for it, so that साबूदाना can still reach a product listed as sago. Without a model, the offline parser recognises only the verbs, postpositions and units listed in the normalisation module, so an unfamiliar verb or construction can produce a wrong draft, and a product word missing from the lexicon cannot be linked to a catalogue that names the product in English. Such a line is left as a custom item at zero price for the user to resolve. Section VI-C measures these effects.

D. Image Ingestion

Two engines can read a photographed slip, and both are free. The first choice is the Gemini vision model, which is instructed to ignore anything crossed out. The fallback is OCR.space, a character-level engine. Both return the same structure (party, phone, document type, line items, total, date and discount), so everything downstream is shared. A post-processing module reconstructs item rows from OCR text and corrects two errors seen on real slips, where the digit "1" came back as the letter "I" and a tab joined two short rows into one. When only the OCR engine is available, the draft is labelled with that engine's name on the confirmation screen, which warns the reviewer that a cancelled line may be present.

E. Inbound Orders by Email and Telegram

Customers can also send orders by email or through a Telegram bot. A background collector polls each configured channel every 30 seconds on its own thread, so an unreachable mail server cannot stall the event worker. Each business is given a private token that routes mail and linked chats to it. Each inbound message is placed in a review queue as a draft, and it is recorded only when a user with write permission accepts it. A count of pending messages, refreshed every 20 seconds, appears in the sidebar and the browser tab title on whatever page is open, so staff see a new order without visiting the inbox.

When two staff members open the same message, the row is locked for update. The second person to accept waits for the first to commit, finds the message already handled, and receives a conflict response, so the order is recorded once. An accepted document stores the channel it came from (Form, AI Text, AI Photo, Email or Telegram), taken from the stored message rather than from the edited draft, so that correcting a draft cannot change its recorded origin.

F. Event Backbone

Every business write publishes its event into an outbox table within the same database transaction, as illustrated in Figure 4. A sale and the record that the sale happened therefore either both exist or neither does. Six event types are defined: SALE_CREATED, PURCHASE_CREATED, STOCK_UPDATED, EXPENSE_ADDED, PAYMENT_RECEIVED and ORDER_CREATED.

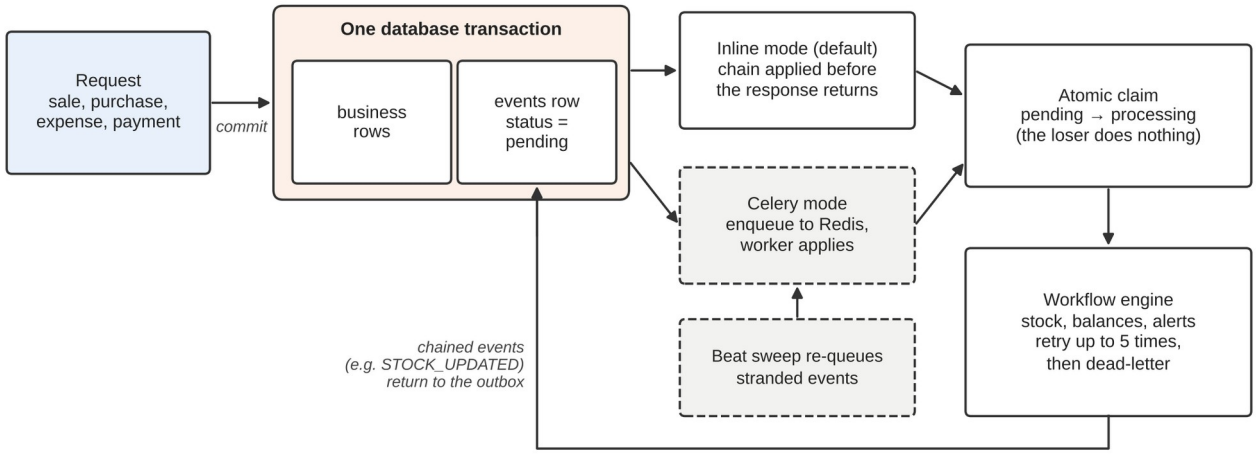


Figure 4: Transactional outbox and event dispatch in inline and Celery modes

A worker claims each pending event with a single atomic update that changes its status from pending to processing and records the time of the claim. It then runs the event's effects and rules inside one transaction. Failed events are retried, and after five attempts they are moved to a dead-letter state. Since every handler is a single transaction, a failed attempt rolls back completely and an automatic retry starts from a clean state. A manual replay is also checked for existing stock movements, so that re-running a sale cannot reduce inventory twice. An event can raise further events: a sale raises SALE_CREATED, processing it moves stock and raises STOCK_UPDATED, and that may trigger a low-stock alert. Chained events inherit the author of the event that started the chain, so the outbox also records who caused each change.

Two dispatch modes are available. In the default inline mode, the whole chain is applied within the request before the response is returned, so stock read immediately after a sale is already correct. In Celery mode, events are handed to Redis and applied by workers that scale independently of the API. The write then returns before its effects exist, which is why this mode is optional. Redis is used only as a broker. If an enqueue fails, the event is already committed as pending, and a periodic sweep re-delivers it. The sweep judges abandonment by the age of the claim rather than the age of the event, since after an outage the entire backlog is old and judging by creation time would hand one event to two workers. Delivery is therefore at-least-once, while application is exactly-once: of two workers given the same event, only one wins the atomic claim and the other does nothing.

G. Workflow Rule Engine

Inventory and balance updates are core effects that always run, so the ledger remains consistent even if a user disables a rule. Configurable behaviour is expressed as WHEN/THEN rules in the ECA style [18]. Each rule names an event type, an optional condition (a field, an operator from greater-than, greater-or-equal, less-than, less-or-equal, equal and not-equal, and a value), and an action. A condition on a field that the event does not carry evaluates to false, so a not-equal rule cannot match by accident. Every evaluation is written to an executions table and shown on the workflow page. Table I lists the rules with which each new business starts.

Table I: Default workflow rules for a new business

Rule	When	Then
Update inventory on sale	SALE_CREATED	update_inventory
Low-stock restock alert	STOCK_UPDATED	restock_alert
Flag high-value expense	EXPENSE_ADDED, amount > 10000	flag_expense
Unpaid sale reminder	SALE_CREATED, payment status = unpaid	notify

H. Business Health Score

The dashboard summarises the state of the business as a score out of 100, composed of four parts that are each scored from 0 to 100. Each mark is labelled Healthy (70 and above), Fair (40 to 69) or At risk (below 40). Every window, threshold and weight is a named constant, and the scoring functions are pure, which allows them to be tested without a database. Table II summarises the parts.

Table II: Components of the business-health score

Part	Weight	What it measures
Cash flow	30%	On-time collection, on-time supplier payment, cash in against cash out (90 days)
Profitability	30%	Net margin after cost of goods and expenses, net of GST (90 days)
Revenue	25%	Last 30 days against the average month of the preceding 90 days
Inventory	15%	Sales-weighted availability (70%) and absence of dead stock (30%)

Cash flow averages up to three measures over the last 90 days: the share of customer credit received by its due date, the share of supplier credit paid by its due date, and cash movement, given by

$$movement = 0.5 + 2.5 \times (cash\ in - cash\ out) / cash\ in \quad (1)$$

held between 0 and 1. Profitability is scored as 40 + 600 × net margin, clipped to the range 0 to 100, so that break-even scores 40 and a 5% margin scores 70. Revenue compares the last 30 days with the average month of the preceding 90 days, giving 75 + 100 × growth × confidence, where confidence is reduced while the baseline contains fewer than twenty sales. Inventory weights availability, measured by recent sales, at 70% and the absence of dead stock at 30%.

Two further rules apply to the overall score. A part with nothing to measure is dropped and the remaining weights are rescaled, and no overall score is shown unless at least half of the weight is backed by data. The overall score also cannot exceed the weakest part by more than 30 points, so one weak area cannot be hidden by three strong ones. With cash flow at 10 and every other part at 100, the weighted average would be 73, but the reported score is 40.

The cash-flow part depends on a payment ledger in which every movement of money is stored with the day it occurred and the way it was made. Due dates are derived from the document date plus the credit days of the document, the party or the business, in that order of precedence. An earlier version of the score compared the amounts owed in each direction, and under it ₹10 lakh of uncollected invoices scored 95. Uncollected invoices are not cash in hand, so the current score counts only money that actually moved, together with the date it moved.

I. Security and Access Control

Passwords are stored using PBKDF2-HMAC-SHA256 with 100,000 iterations [25], and sessions use a signed JSON Web Token [26] held in an httpOnly cookie. Access follows a role-based model [27] with four roles and seven coarse permissions. Two roles, superuser and admin, operate at platform level, and two, business owner and employee, belong to a single business. An admin can configure a business but holds no permission to create, change or remove its sales, purchases, expenses, products or parties, and an employee may add records but cannot cancel, delete or back-date them. A request for another tenant's resource returns 404, not 403, so it reveals nothing about whether that tenant exists. Failed sign-ins are throttled per account and per IP address, with the state kept in PostgreSQL so that several API processes share it. Invitation tokens are stored only as SHA-256 hashes. For card payments, only the last four digits are accepted, and a longer number is refused rather than trimmed.

VI. EVALUATION AND RESULTS

A. Method

Every evaluation input has a gold label written by a person before any system processed it. A harness runs each input through the same functions that the application uses and compares the output with its label field by field. Precision, recall and F1 are reported per field, following standard information-extraction practice [28]. A wrong answer is counted both as a false positive and as a false negative, which penalises a fabricated customer more heavily than a missing one. Exact match is the share of inputs for which every scored field was correct, which corresponds to the share of drafts a shopkeeper could accept without editing. Offline results were measured on the final version of the code. Model results come from the most recent runs against the hosted service, made in September for the original notes and the slips and in November for the out-of-vocabulary set, before the final adjustments to the prompt and matcher described in Section V.

B. Typed Notes

The note set contains 70 inputs across five language variants. The heuristic parser runs offline, and the model engine is Gemini 3.5 Flash Lite. Results are given in Table III and Table IV.

Table III: Results on typed notes (n = 70); per-field F1 in %

Engine	Exact	Type	Party	Items	Amt.	Cat.
Heuristic	87.1%	100.0	94.8	93.7	95.2	100.0
Gemini	97.1%	100.0	100.0	96.8	100.0	100.0

Table IV: Exact match by language variant

Variant	n	Heuristic	Gemini
English	18	66.7%	94.4%
Hindi, Latin letters	18	83.3%	94.4%
Hindi, Devanagari	11	100.0%	100.0%
Kannada, Latin letters	12	100.0%	100.0%
Kannada script	11	100.0%	100.0%

On well-formed notes the offline parser keeps pace with the model. It loses ground on messy ones (no verb, no capitals, typing errors, two items on one line), and those account for its lower scores in English and in Hindi typed in Latin letters. Both native scripts reach 100% with either engine. The model reads them directly, and the offline parser reaches the same result once the normalisation stage has rewritten the note into English word order, so on these notes the two engines cannot be told apart. The accuracy of the two engines is compared in Figure 5.

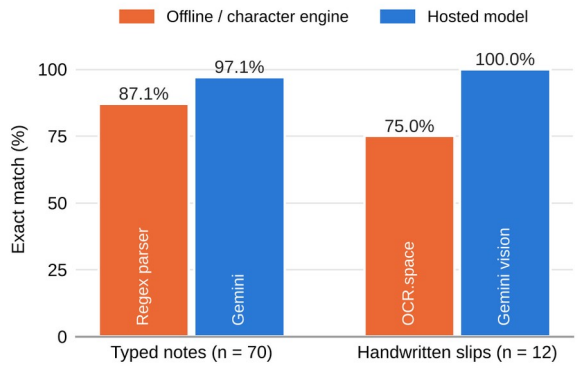


Figure 5: Exact-match accuracy of each engine on typed notes and handwritten slips

C. Out-of-Vocabulary Notes

The note set above was written by the author of the parser and mostly uses words the system already knows. A second set of 24 notes, six in each Hindi and Kannada variant, was written to test what lies outside that vocabulary. Half of them use verbs and constructions that the normalisation module does not list, such as "de diya" (gave), "le gaye" (took away), "kotte" (gave) and "bantu" (came), with parties and products the shop already has. The other half name products that are absent from the lexicon. These are scored against an extended catalogue with six additional products, four of them named in English (sago, raisins, camphor and dates) and two named with the Indian word (ragi flour and jowar). Table V summarises the results.

Table V: Out-of-vocabulary notes, fully correct drafts (*earlier run, before the final prompt and matcher; one note fell back after an HTTP 503 and is excluded)

Engine	Unusual grammar	Unlisted products	Exact
Heuristic	4 / 12	3 / 12	29.2% (7/24)
Gemini*	10 / 12	1 / 11	47.8% (11/23)

The offline parser produced a fully correct draft for 7 of the 24 notes (29.2%), compared with 87.1% on the original set. It usually identified the event type correctly (F1 of 95.8), but the F1 for parties fell to 69.8 and for items to 62.5. Unfamiliar grammar mainly disrupted party detection. An honorific ("ji"), a group marker ("wale") or a Kannada pronoun ("avarige", "avru") left the customer missing or misread, and a note about goods arriving from a supplier was recorded as a sale. Of the twelve notes naming unlisted products, three were fully correct: the jowar note and the two ragi flour notes, where the catalogue uses the Indian word and the qualifier rule chose ragi flour over wheat flour. Where the catalogue used an English name, the product remained as unmatched text in the note's own script, since the offline path has no means of translating it.

The language model fared considerably better on grammar. Its results on this set come from the earlier run noted in Section VI-A. During that run one call was refused by the provider with a temporary overload error (HTTP 503) and that note was processed by the offline parser instead, so the model's figures cover the 23 notes it actually handled. It produced a correct draft for 10 of the 12 notes with unfamiliar grammar and for 11 of the 23 notes overall (47.8%), with an event-type F1 of 100 and a party F1 of 87.5. Both of its grammar failures arose after extraction, in the matching step. In one, the model returned the customer as "अनीता जी" with the honorific attached, which the matcher did not resolve to Anita Stores. In the other, it correctly identified a purchase from "ಸನ್ರೈಸ್ ಹೋಲ್ಸೇಲ್", but the English trade name written in Kannada script did not transliterate closely enough to "Sunrise Wholesale"; the offline parser failed on "ಎಬಿಸಿ ಸಪ್ಲೈಯರ್ಸ್" for the same reason.

On unlisted products, that run returned product words in the script of the note, and their matching therefore depended on the lexicon in the same way as for the offline parser. The model was fully correct on only 1 of the 11 such notes it handled, a figure that does not reflect the English product names now requested in the prompt.

D. Handwritten Order Slips

Twelve real photographs of handwritten slips were evaluated. Table VI reports the results.

Table VI: Results on real handwritten slips (n = 12); F1 in %; clean and corrected (Corr.) slips counted separately

Engine	Exact	Party	Phone	Items	Clean	Corr.
Vision	100.0%	100.0	100.0	100.0	9 / 9	3 / 3
OCR.space	75.0%	100.0	100.0	91.7	9 / 9	0 / 3

Both engines read the handwriting accurately, with perfect party and phone scores, and the character engine got all nine clean slips right. The whole difference lies in three slips where the writer had struck out an item and rewritten it. On one, a crossed-out "Coffee" was followed by "1 kg Coffee", so a literal reading billed two kilograms for one. On another, "Wheat" was struck through before "Atta", and the literal product became "Wheat Atta". On the third, a cancelled item was read as ordered. The vision model got all three right and character OCR got none. Tuning cannot fix this, because post-processing has no way to recover a mark that never made it into the extracted text. A quarter of an unplanned sample (three slips out of twelve) contained such a correction, so the case appears to be common. For this reason the vision engine is tried first, and the fallback engine is named on the confirmation screen.

E. Consistency

Five notes were run three times each to test whether identical input produces an identical draft. At the model's default temperature, only three of the five notes gave the same draft on every run. In one failure, a Kannada case ending remained attached to the customer's name and matched no customer. In another, an order of rice and sugar was occasionally collapsed into a single generic item. With the temperature fixed at zero, all five notes gave the same draft on every run, and exact-match accuracy stayed at 97.1%.

F. Extraction Latency

The median time from a typed note to a draft through the model, including catalogue matching, was 2.2 s. A photograph took 1.4 s through the vision model and 1.9 s through OCR.space, the latter figure including image re-encoding. The 90th percentile for text notes was 16.8 s. That figure includes the waits of the harness, which retried with back-off whenever the free tier refused a request, so it reflects the rate limit more than the model.

G. Throughput Under Load

A Locust script modelled activity on a shop floor, with roughly eight reads for every write, weighted towards the dashboard and document lists, and with sales, expenses and Smart Input parses in between. Two hundred concurrent users were run against a business holding 4,000 sales on a 16-core laptop. As Table VII and Figure 6 show, each fix exposed the next limit.

Table VII: Load test with 200 concurrent users and 4,000 sales

Step	Requests	Failed	Req/s	Median (ms)	p95 (ms)
Baseline	129	19.4%	3.5	2,200	33,000
Larger pool	430	19.3%	9.8	6,500	31,000
+ 4 workers	994	5.4%	16.8	3,500	19,000
+ SQL aggr.	6,301	0%	106.3	33	150

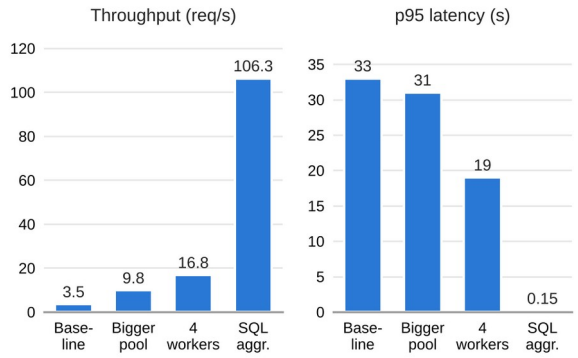


Figure 6: Throughput and 95th-percentile latency after each optimisation step (200 users)

The first limit was the connection pool. SQLAlchemy's default of five connections plus ten overflow capped concurrency at fifteen requests, and the 33-second 95th percentile matched the 30-second pool timeout followed by an error. The pool was enlarged to 20 plus 40 with a 10-second timeout, so that a starved request now fails quickly. The second limit was that a single Python process uses one core, while each sign-in deliberately spends tens of milliseconds on PBKDF2. Running four worker processes addressed this. The third limit was the dashboard, which loaded every document into memory and summed it in Python, taking 1.40 s per request. Rewriting it as SQL aggregates reduced the time to 0.24 s, and the outputs were compared against the previous implementation on two tenants to confirm they were identical. Indexes on the tenant column contributed as well: on 200,000 sales across 50 businesses, the sales-list query fell from 10.50 ms with a parallel sequential scan to 0.067 ms with an index scan.

H. Test Suite

The backend has 484 automated tests across seventeen layers. These cover domain arithmetic, all 73 HTTP routes, role permissions, pagination, both dispatch modes, Hindi and Kannada input, the AI path with the model replaced by a stub, inbound email and Telegram processing, payments, and the health score, which alone has 55 tests. The suite clears every API key and makes no network calls. Passing tests did not guarantee correct extractions, though. When 364 tests were passing, the evaluation still exposed four defects: an expense category vocabulary that differed between the two engines (fixing it raised exact match from 90.0% to 97.1%), a tab character that consumed a phone number, a name and phone number merged into one line, and a fault in the harness itself, where 300 and 300.0 were

compared as unequal text. On the first real run, the harness also found that 38 of the 70 notes had silently fallen back to the heuristic parser because of the rate limit, and it now reports such substitutions.

VII. DISCUSSION AND LIMITATIONS

Four observations follow from the results. First, on well-formed notes, a deterministic offline parser with careful normalisation comes close to a hosted model, so the system stays useful without any external service. That finding does not extend to unfamiliar phrasing, on which the offline parser's exact-match accuracy fell to 29.2% while the model, in its earlier run, reached 47.8%. Second, most of the model's remaining errors occurred in catalogue matching rather than in language understanding, which places the main weakness of the multilingual pipeline in its grounding stage. Third, the choice between character OCR and a vision model matters mainly for in-place corrections, and the confirmation step, labelled by engine, limits the risk that remains. Finally, on a single laptop, the modular monolith's throughput was held back by connection pooling, process count and query design, each of which was a configuration or code change.

The evaluation has clear limitations. The typed notes are synthetic and were written by the author of the parser. The first fifty all scored 100% on the heuristic parser, which mostly shows that the notes matched their author's expectations; twenty deliberately messier notes added later brought the score below 90%. The slips are real photographs, but all of them were written by one person in neat block capitals and photographed in good light. Each gold label was written once by a single annotator, so no inter-annotator agreement can be reported. Latency was measured on single calls to an idle machine, and the behaviour of the AI path when many shopkeepers submit notes in the same minute is unknown, with the free tier's rate limit the likely constraint.

The language support is also narrower than the native-script rows of Table IV suggest. Those notes were written by the author of the normalisation rules and the lexicon, and they mostly use words that both already contain. The out-of-vocabulary test shows that the offline path depends on a hand-maintained vocabulary of 94 product concepts and cannot translate a word outside it. The English product names requested from the model are intended to reduce that dependence when a model is configured, although their effect on this set has not yet been measured. Without one, a shop selling goods outside the lexicon would need to extend it or rely on the confirmation step to catch unmatched lines.

The health score is limited by its inputs. It cannot see the bank balance or the owner's drawings, cost of goods uses each product's current purchase price rather than the historical one, GST collected is counted as cash in while its remittance happens outside the system, and a festival month makes the following month look like a decline. When a model key is configured, notes and images are sent to the provider, and retention is governed by the provider's terms. Without a key, nothing leaves the machine.

VIII. CONCLUSION

Many small businesses take orders informally, and software that depends on form entry leaves much of that trade unrecorded. SmartSME accepts notes, photographs, emails and chat messages in English, Hindi and Kannada, and turns each into a draft that a person confirms before anything is written. A transactional outbox in PostgreSQL keeps every record consistent with its effects without a separate broker, and the health score, computed from a dated payment ledger, summarises the business using money that actually moved.

Offline extraction produced fully correct drafts for 87.1% of typed notes and model-based extraction for 97.1%. A vision model read all twelve real slips correctly, including the three with corrections, and the system served 106.3 requests per second under 200 simulated users. On notes written outside its vocabulary, however, the offline parser was fully correct only 29.2% of the time and an earlier run of the model 47.8%, with most remaining errors traced to catalogue matching. Because the typed notes were written by the parser's author and the slips by a single writer, these results need to be confirmed by a small deployment in working shops before the platform is used more widely.

Several directions for future work follow. The most pressing is evidence from real use: a deployment in a handful of shops, with notes and slips written by actual customers and labelled independently by two annotators, would show how the system performs for people other than its authors. Vocabulary is the second. Every time a user fixes an unmatched line on the confirmation screen, the system observes a pairing between a word and a catalogue product, and storing those pairings for each shop would allow the lexicon to grow through use. Small language models running on the shop's own device could widen the offline path's language coverage without sending notes to an outside provider. Voice notes, which are common in chat-based ordering, are a natural next input, and a WhatsApp channel could sit alongside email and Telegram. The concurrent behaviour of the AI path, and the current model prompt on the out-of-vocabulary set, also remain to be measured.

The analytical side offers further scope. Once a year of data exists, the revenue score could compare each month with the same month of the previous year, which would remove the distortion caused by festival seasons. A dated payment ledger could also, with the owner's consent, support credit assessment for small firms that lack formal financial statements, provided that such use complies with the Digital Personal Data Protection Act, 2023 [29].

REFERENCES

[1] Press Information Bureau, Ministry of Micro, Small and Medium Enterprises, "Contribution of MSMEs to the GDP," press release, New Delhi, Jul. 22, 2024. [Online]. Available: https://pib.gov.in/PressReleasePage.aspx?PRID=2035073

[2] National Statistics Office, Ministry of Statistics and Programme Implementation, "Annual Survey of Unincorporated Sector Enterprises (ASUSE) 2023-24," press note, New Delhi, Jan. 29, 2025.

[3] Government of India, "The Micro, Small and Medium Enterprises Development Act, 2006," Act No. 27 of 2006, Ministry of Law and Justice, 2006.

[4] P. Vashisht, MSME Digitalisation in India: Current Status and Challenges. New Delhi, India: Research and Information System for Developing Countries (RIS), 2025.

[5] M. Haddara and O. Zach, "ERP systems in SMEs: A literature review," in Proceedings of the 44th Hawaii International Conference on System Sciences (HICSS), pp. 1-10, 2011.

[6] M. Al-Mashari, A. Al-Mudimigh, and M. Zairi, "Enterprise resource planning: A taxonomy of critical factors," European Journal of Operational Research, vol. 146, no. 2, pp. 352-364, 2003.

[7] K. Bali, J. Sharma, M. Choudhury, and Y. Vyas, "'I am borrowing ya mixing?' An analysis of English-Hindi code mixing in Facebook," in Proceedings of the First Workshop on Computational Approaches to Code Switching, pp. 116-126, 2014.

[8] S. Khanuja, S. Dandapat, A. Srinivasan, S. Sitaram, and M. Choudhury, "GLUECoS: An evaluation benchmark for code-switched NLP," in Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, pp. 3575-3585, 2020.

[9] R. Smith, "An overview of the Tesseract OCR engine," in Proceedings of the Ninth International Conference on Document Analysis and Recognition (ICDAR), vol. 2, pp. 629-633, 2007.

[10] T. Brown et al., "Language models are few-shot learners," in Advances in Neural Information Processing Systems, vol. 33, pp. 1877-1901, 2020.

[11] Z. Ji et al., "Survey of hallucination in natural language generation," ACM Computing Surveys, vol. 55, no. 12, pp. 1-38, 2023.

[12] P. Helland, "Life beyond distributed transactions: An apostate's opinion," in Proceedings of the Third Biennial Conference on Innovative Data Systems Research (CIDR), Asilomar, CA, USA, 2007.

[13] G. Hohpe and B. Woolf, Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions. Boston, MA, USA: Addison-Wesley, 2003.

[14] G. Mühl, L. Fiege, and P. Pietzuch, Distributed Event-Based Systems. Berlin, Germany: Springer, 2006.

[15] M. Fowler, "Event sourcing," martinfowler.com, Dec. 2005. [Online]. Available: https://martinfowler.com/eaaDev/EventSourcing.html

[16] C. Richardson, Microservices Patterns: With Examples in Java. Shelter Island, NY, USA: Manning, 2018.

[17] M. Kleppmann, Designing Data-Intensive Applications. Sebastopol, CA, USA: O'Reilly Media, 2017.

[18] U. Dayal et al., "The HiPAC project: Combining active databases and timing constraints," ACM SIGMOD Record, vol. 17, no. 1, pp. 51-70, 1988.

[19] C. L. Forgy, "Rete: A fast algorithm for the many pattern/many object pattern match problem," Artificial Intelligence, vol. 19, no. 1, pp. 17-37, 1982.

[20] Y. Xu, M. Li, L. Cui, S. Huang, F. Wei, and M. Zhou, "LayoutLM: Pre-training of text and layout for document image understanding," in Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 1192-1200, 2020.

[21] G. Kim et al., "OCR-free document understanding transformer," in Computer Vision - ECCV 2022, Lecture Notes in Computer Science, vol. 13688. Cham, Switzerland: Springer, 2022.

[22] Gemini Team, Google, "Gemini: A family of highly capable multimodal models," arXiv preprint arXiv:2312.11805, 2023.

[23] "indic-transliteration: Transliteration tools to convert text in one Indic script encoding to another," Python Package Index, version 2.3. [Online]. Available: https://pypi.org/project/indic-transliteration/

[24] E. I. Altman, "Financial ratios, discriminant analysis and the prediction of corporate bankruptcy," The Journal of Finance, vol. 23, no. 4, pp. 589-609, 1968.

[25] K. Moriarty, B. Kaliski, and A. Rusch, "PKCS #5: Password-based cryptography specification version 2.1," RFC 8018, Internet Engineering Task Force, Jan. 2017.

[26] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, Internet Engineering Task Force, May 2015.

[27] R. S. Sandhu, E. J. Coyne, H. L. Feinstein, and C. E. Youman, "Role-based access control models," IEEE Computer, vol. 29, no. 2, pp. 38-47, 1996.

[28] N. Chinchor, "MUC-4 evaluation metrics," in Proceedings of the Fourth Message Understanding Conference (MUC-4), pp. 22-29, 1992.

[29] Government of India, "The Digital Personal Data Protection Act, 2023," Act No. 22 of 2023, Ministry of Law and Justice, 2023.

★★★